

Question 1:**code:**

```
library(ggplot2)

# Create dataset

employee <- data.frame(

  Employee_ID = paste0("E", sprintf("%03d", 1:20)),

  Department = c(rep("IT",5), rep("HR",5), rep("Finance",5), rep("Marketing",5)),

  Salary = c(42000,45000,48000,51000,55000,

             38000,40000,42000,46000,50000,

             47000,49000,52000,58000,65000,

             36000,39000,43000,47000,72000)

)

# 1. ECDF Plot

ggplot(employee, aes(x = Salary)) +

  stat_ecdf(geom = "step") +

  labs(title = "ECDF of Employee Salaries",

       x = "Salary (₹)", y = "Cumulative Proportion") +

  theme_minimal()

# 2. Q-Q Plot

ggplot(employee, aes(sample = Salary)) +

  stat_qq() +

  stat_qq_line() +

  labs(title = "Q-Q Plot of Employee Salaries",

       x = "Theoretical Quantiles",

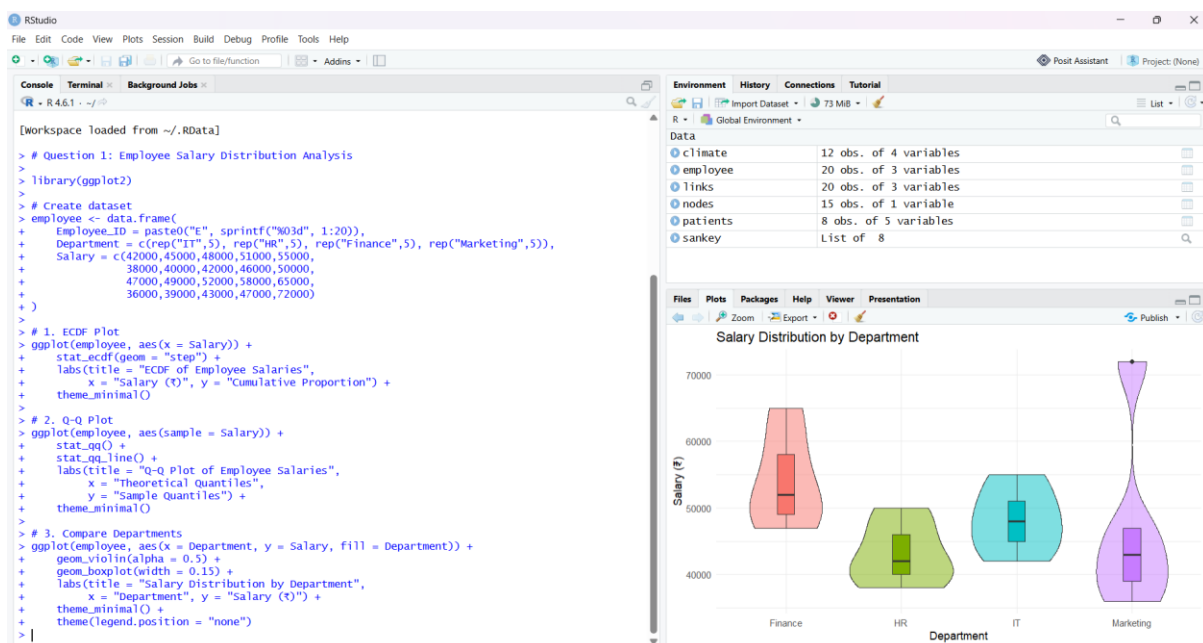
       y = "Sample Quantiles") +

  theme_minimal()
```

3. Compare Departments

```
ggplot(employee, aes(x = Department, y = Salary, fill = Department)) +  
  geom_violin(alpha = 0.5) +  
  geom_boxplot(width = 0.15) +  
  labs(title = "Salary Distribution by Department",  
        x = "Department", y = "Salary (₹)") +  
  theme_minimal() +  
  theme(legend.position = "none")
```

Output:



Question 2:

Code:

```
library(ggplot2)  
  
# Create dataset  
student <- data.frame(  
  Student_ID = paste0("S", sprintf("%03d", 1:20)),  
  Course = c(rep("Python",5), rep("Statistics",5),  
              rep("DataViz",5), rep("MachineLearning",5)),
```

```

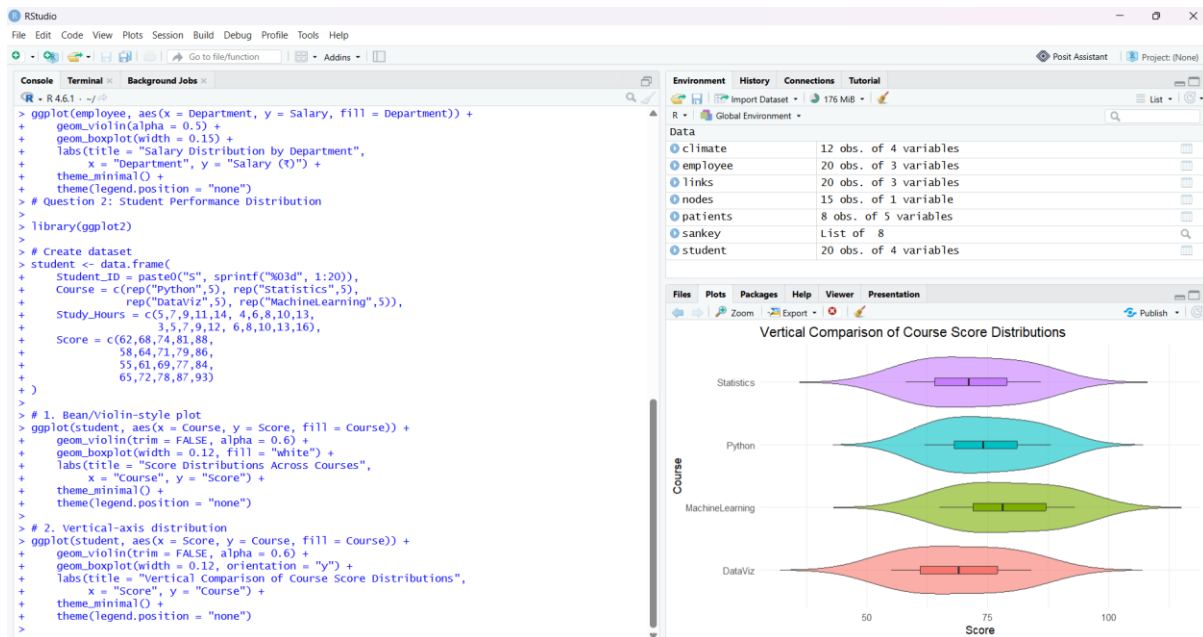
Study_Hours = c(5,7,9,11,14, 4,6,8,10,13,
                3,5,7,9,12, 6,8,10,13,16),
Score = c(62,68,74,81,88,
          58,64,71,79,86,
          55,61,69,77,84,
          65,72,78,87,93)
)

# 1. Bean/Violin-style plot
ggplot(student, aes(x = Course, y = Score, fill = Course)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.12, fill = "white") +
  labs(title = "Score Distributions Across Courses",
       x = "Course", y = "Score") +
  theme_minimal() +
  theme(legend.position = "none")

# 2. Vertical-axis distribution
ggplot(student, aes(x = Score, y = Course, fill = Course)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.12, orientation = "y") +
  labs(title = "Vertical Comparison of Course Score Distributions",
       x = "Score", y = "Course") +
  theme_minimal() +
  theme(legend.position = "none")

```

Output:



Question 3:

Code:

```
library(ggplot2)
```

```
# Create dataset
```

```
purchase <- data.frame(
```

```
  Customer_ID = paste0("C", sprintf("%03d", 1:12)),
```

```
  Region = c("North","North","North",
```

```
            "South","South","South",
```

```
            "East","East","East",
```

```
            "West","West","West")),
```

```
  Product = rep(c("Laptop","Phone","Tablet"),4),
```

```
  Purchase_Type = c("Online","Online","Store",
```

```
                  "Store","Online","Store",
```

```
                  "Online","Store","Online",
```

```
                  "Store","Online","Store"),
```

```
  Count = c(25,35,20,30,40,25,28,32,18,22,38,27)
```

```
)
```

```
# 1. Pie Chart
```

```
product_total <- aggregate(Count ~ Product, data = purchase, sum)
ggplot(product_total, aes(x = "", y = Count, fill = Product)) +
  geom_col() +
  coord_polar("y") +
  labs(title = "Overall Purchase Proportion by Product") +
  theme_void()
```

2. Side-by-side Bar Chart

```
ggplot(purchase, aes(x = Region, y = Count, fill = Product)) +
  geom_col(position = "dodge") +
  labs(title = "Product Purchases Across Regions",
       x = "Region", y = "Purchase Count") +
  theme_minimal()
```

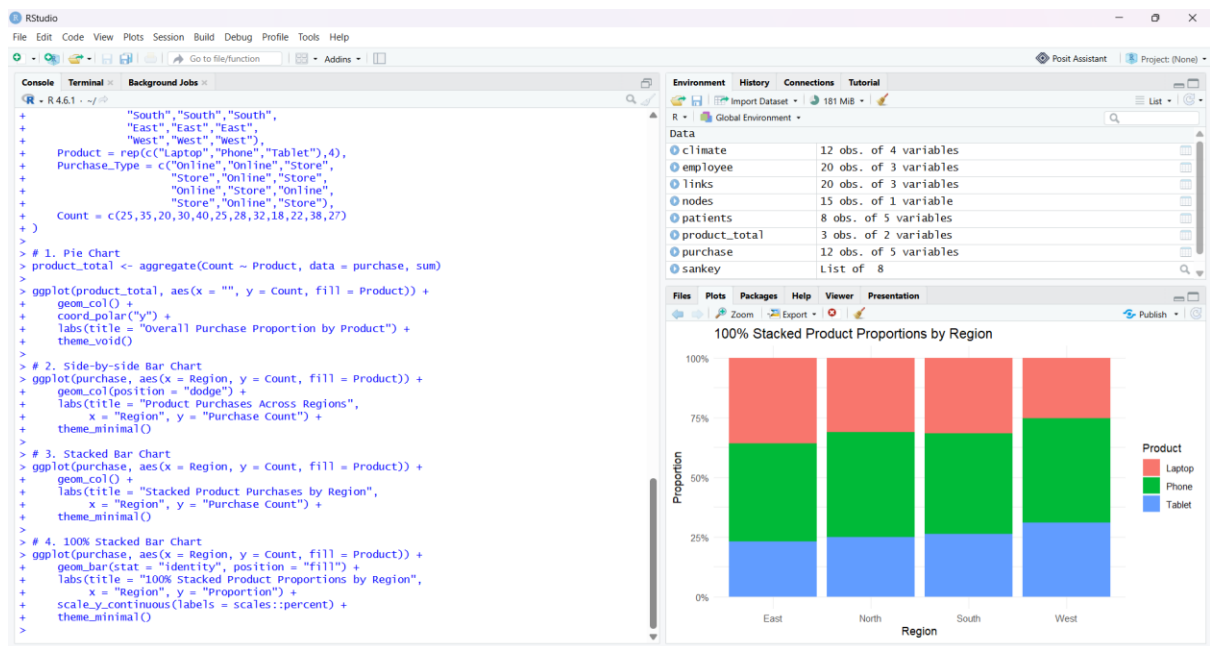
3. Stacked Bar Chart

```
ggplot(purchase, aes(x = Region, y = Count, fill = Product)) +
  geom_col() +
  labs(title = "Stacked Product Purchases by Region",
       x = "Region", y = "Purchase Count") +
  theme_minimal()
```

4. 100% Stacked Bar Chart

```
ggplot(purchase, aes(x = Region, y = Count, fill = Product)) +
  geom_bar(stat = "identity", position = "fill") +
  labs(title = "100% Stacked Product Proportions by Region",
       x = "Region", y = "Proportion") +
  scale_y_continuous(labels = scales::percent) +
  theme_minimal()
```

Output:



Question 4:

Code:

```
# Region -> Category -> Subcategory
```

```
library(treemap)
```

```
library(plotly)
```

```
# Create dataset
```

```
sales <- data.frame(
```

```
  Region = c(
```

```
    "North", "North", "North", "North",
```

```
    "South", "South", "South", "South",
```

```
    "East", "East", "East", "East",
```

```
    "West", "West", "West", "West"
```

```
),
```

```
  Category = c(
```

```
    "Electronics", "Electronics", "Furniture", "Furniture",
```

```
    "Electronics", "Electronics", "Furniture", "Furniture",
```

```
    "Electronics", "Electronics", "Furniture", "Furniture",
```

```
    "Electronics", "Electronics", "Furniture", "Furniture"
```

```

),
Subcategory = c(
  "Laptop","Phone","Chair","Desk",
  "Laptop","Phone","Chair","Desk",
  "Laptop","Phone","Chair","Desk",
  "Laptop","Phone","Chair","Desk"

```

```

),
Sales = c(
  52000,38000,22000,18000,
  46000,42000,25000,21000,
  58000,35000,19000,23000,
  49000,44000,27000,20000

```

```

)
```

```

)
```

```
# 1. TREEMAP
```

```

treemap(
  sales,
  index = c("Region", "Category", "Subcategory"),
  vSize = "Sales",
  type = "index",
  title = "Company Sales: Region → Category → Subcategory"

```

```

)
```

```
# 2. SUNBURST
```

```
# Create unique IDs for each hierarchy level
```

```
region <- aggregate(Sales ~ Region, data = sales, sum)
```

```

category <- aggregate(
  Sales ~ Region + Category,
  data = sales,
  sum

```

```

)
subcategory <- sales
# Region level
region_data <- data.frame(
  id = region$Region,
  label = region$Region,
  parent = "",
  value = region$Sales
)
# Category level
category_data <- data.frame(
  id = paste(category$Region, category$Category, sep = "-"),
  label = category$Category,
  parent = category$Region,
  value = category$Sales
)
# Subcategory level
subcategory_data <- data.frame(
  id = paste(
    sales$Region,
    sales$Category,
    sales$Subcategory,
    sep = "-"
  ),
  label = sales$Subcategory,
  parent = paste(
    sales$Region,
    sales$Category,
    sep = "-"

```

```

    ),
    value = sales$Sales
  )
# Combine all three levels
sunburst_data <- rbind(
  region_data,
  category_data,
  subcategory_data
)
# Create sunburst chart
plot_ly(
  data = sunburst_data,
  ids = ~id,
  labels = ~label,
  parents = ~parent,
  values = ~value,
  type = "sunburst",
  branchvalues = "total"
) %>%
  layout(
    title = "Company Sales Sunburst Chart"
  )
# 3. FIND LARGEST REGION
largest_region <- region[which.max(region$Sales), ]
print("Largest Region:")
print(largest_region)
# 4. FIND LARGEST CATEGORY
category_total <- aggregate(
  Sales ~ Category,

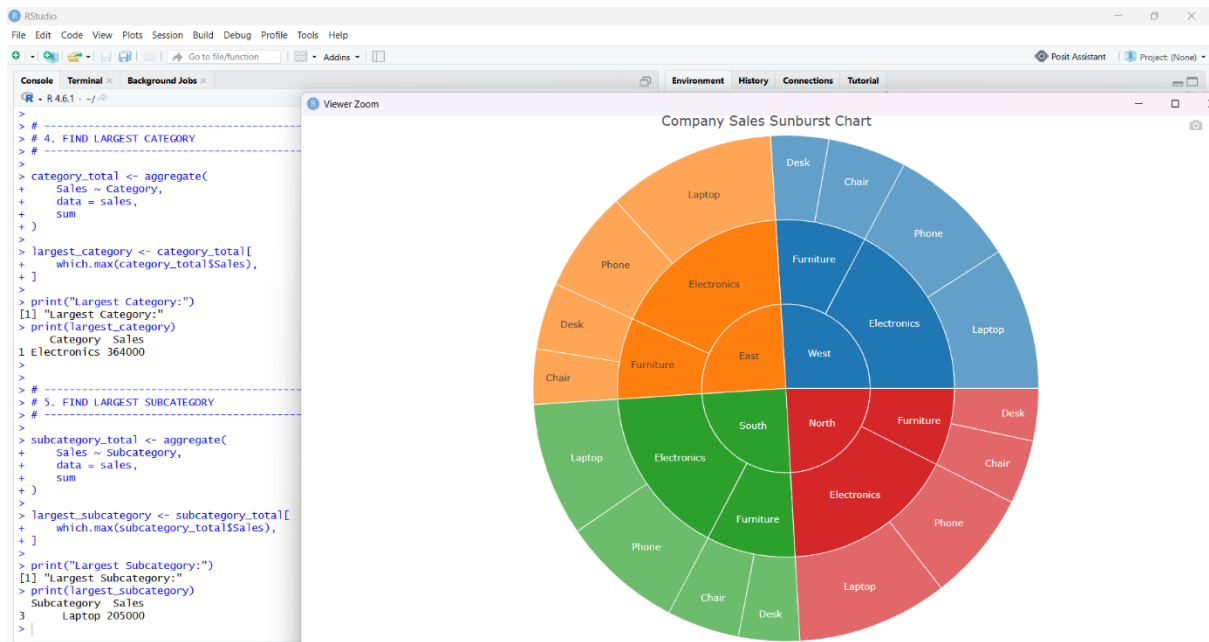
```

```

    data = sales,
    sum
)
largest_category <- category_total[
  which.max(category_total$Sales),
]
print("Largest Category:")
print(largest_category)
# 5. FIND LARGEST SUBCATEGORY
subcategory_total <- aggregate(
  Sales ~ Subcategory,
  data = sales,
  sum
)
largest_subcategory <- subcategory_total[
  which.max(subcategory_total$Sales),
]
print("Largest Subcategory:")
print(largest_subcategory)

```

Output:



Question 5:

Code:

```
library(ggplot2)
```

```
library(ggalluvial)
```

```
# Create dataset
```

```
career <- data.frame(
```

```
  Student_Group = paste0("G", 1:12),
```

```
  Specialization = c("Data Analytics","Data Analytics","Data Analytics",
```

```
    "AI","AI","AI",
```

```
    "Cyber Security","Cyber Security","Cyber Security",
```

```
    "Cloud Computing","Cloud Computing","Cloud Computing"),
```

```
  Internship = c("Yes","Yes","No",
```

```
    "Yes","Yes","No",
```

```
    "Yes","Yes","No",
```

```
    "Yes","Yes","No"),
```

```
  Placement_Sector = c("IT","Finance","IT",
```

```
    "IT","Healthcare","IT",
```

```
    "IT","Finance","Government"),
```

```

      "IT", "Finance", "Government"),
Students = c(18,12,8,20,10,6,16,9,7,15,8,5)
)
# 1. Sankey-style Alluvial Diagram
ggplot(career,
  aes(axis1 = Specialization,
    axis2 = Internship,
    axis3 = Placement_Sector,
    y = Students)) +
geom_alluvium(aes(fill = Specialization), alpha = 0.7) +
geom_stratum() +
geom_text(stat = "stratum", aes(label = after_stat(stratum))) +
scale_x_discrete(limits = c("Specialization",
  "Internship",
  "Placement Sector"),
  expand = c(.1, .1)) +
labs(title = "Student Career Flow",
  x = "Career Stage",
  y = "Number of Students") +
theme_minimal()

```

2. Parallel Sets Style Visualization

```

ggplot(career,
  aes(axis1 = Specialization,
    axis2 = Internship,
    axis3 = Placement_Sector,
    y = Students)) +
geom_alluvium(aes(fill = Internship), alpha = 0.7) +
geom_stratum() +
geom_text(stat = "stratum", aes(label = after_stat(stratum))) +

```

```

scale_x_discrete(limits = c("Specialization",
                             "Internship",
                             "Placement Sector"),
                 expand = c(.1, .1)) +

labs(title = "Parallel Sets: Student Career Path",
     x = "Variables",
     y = "Number of Students") +

theme_minimal()

```

Output:

